Graduation Project 2025 Problem Formulation

Designing secure software for automotive electronic control units using memory safe languages and hardware security.

Project Goals:

1. Designing a real-time operating system (RTOS) for automotive ECUs.
2. Ensure lightweight dynamically loaded multi-tasking or multi-threading
3. Privilege levels in terms of code and memory.
    a. Core Operating System ⇒ can be in unsafe Rust (safe Rust is preferred)
    b. Drivers ⇒ Must be in safe Rust
    c. Threads/Tasks ⇒ can be in unsafe Rust (Safe Rust is preferred)
4. Drivers mediate access to peripherals among threads/tasks and the core OS.
    a. Ensure memory isolation among threads/tasks!! (this is strict)
    b. A driver must not mix memory objects from different threads/tasks!
    c. A driver may share memory with the core OS, but under strict governance (you must explain why)
5. Implement:
    a. Memory sharing and IPC among threads
    b. Dynamic memory allocation for threads. Recall, no memory sharing among threads!
    c. Memory Protection Unit (MPU)-based isolation.
6. Recall:
    a. the MPU is coarse-grained,
        i. Implementing finer-grained isolation is a plus but not required.
    b. The MPU is almost useless in privileged mode execution
        i. How can we make sure it still has meaning (e.g. when drivers are executing in privileged mode, the MPU offers no protection (implicitly, since any privileged code can modify the MPU)).
        ii. Can you look in Debug hardware tools to solve this problem?